

SSRF — Interview Prep Notes

1. Logical description

- App accepts user-influenced URL/host/path, uses it server-side to make an outbound request
- No/insufficient validation of destination → attacker controls where trusted server sends request
- Root taint model: source (input) → sink (HTTP client) → sanitizer (allowlist check) — same as IDOR/PathTraversal/SQLi taint pattern
- Request inherits server's network trust position → reaches internal-only resources

2. OWASP / CWE

- CWE-918 (SSRF)
- OWASP Top 10 2021: A10:2021 – Server-Side Request Forgery (dedicated category, new in 2021)
- Related: CWE-611 (XXE, common delivery vector)

3. Variants

By response visibility

- Basic/in-band — response reflected
- Semi-blind — partial signal (status/timing/error)
- Blind — no signal, needs OOB confirmation

By vector

- Regular (direct URL control)
- Parser-confusion (`user@host` , `host#frag`)
- DNS rebinding (validate-time IP ≠ request-time IP)
- Open-redirect based (validated hop → attacker redirect → internal)
- File-format driven (XXE, SVG, PDF renderers, image libs)
- Protocol smuggling (`gopher://` , `file://` , `dict://`)
- Second-order (stored URL, triggered later)

By target

- Cloud metadata (169.254.169.254)
- Internal service/API
- Loopback/localhost
- Port-scan recon (semi-blind)

4. Attack surface / entry points

- Explicit params: `url=` , `uri=` , `path=` , `dest=` , `redirect=` , `webhook=` , `callback=` , `src=` , `image=`
- Feature-driven: webhooks, import-by-URL, link preview/unfurl, PDF/doc generation (headless Chrome/wkhtmltopdf), screenshot services, file conversion
- Format-driven: XXE (XML/SOAP/DOCX/XLSX/SAML), SVG `xlink:href` , EXIF/image parsers

- Auth/federation: OAuth `redirect_uri`, SAML metadata/ACS URL, OIDC discovery
- Integration configs: "connect your own instance", health-check URLs
- Headers/body: Host header, POST body URL fields, GraphQL URL-typed args (often missed)
- Recon: grep gau/waybackurls/katana inventory for param patterns; check Swagger/GraphQL introspection

5. Testing workflow (phase-based)

Phase	Verify	Manual	Burp	ZAP	Nuclei	Payloads
1. Surface mapping	Find fetch-capable features	Walk app, check feature docs	Site map + Param Miner, search history for keywords	Spider/AJAX spider, Search tab regex	httpx to filter live URLs, then <code>-tags ssrf</code> fingerprint templates	param name wordlist
2. Baseline	Normal behavior of feature	Submit legit external URL, note resp/time/status	Repeater, record baseline	Manual Request Editor	N/A (manual)	known-good URL
3. External redirect (sink exists?)	Server makes outbound req at all	Own server + log tail / nc listener	Insert Collaborator payload, check Collaborator tab	OAST add-on, check OAST tab	<code>{{interactsh-url}}`, <code>-tags ssrf</code> auto-polls</code>	Collaborator/interactsh unique subdomain
4. Visibility classification	Basic/semi-blind/blind	Diff response vs baseline	Compare Repeater resp	Compare History	Auto via matchers	n/a
5. Internal redirection	Trust boundary enforced?	Manual submit internal IPs/ports	Repeater + Intruder sniper on internal IP/port list	Fuzzer tab, same payload list	Built-in SSRF templates include 127.0.0.1, 169.254.169.254, common ports	<code>127.0.0.1, localhost, [::1], internal ports (22,3306,6379,9200)</code>
6. Bypass reasoning	What/when validated	Reason then test encodings	Intruder payload processing (encode/hash)	Fuzzer with encoded payload list	Custom YAML for org-specific ranges	decimal/octal/hex IP, IPv6-mapped, open-redirect chain, DNS rebinding domain
7. Protocol scope	Scheme restricted?	Try alt schemes	Repeater	Manual editor	Custom template	<code>gopher://, file://, dict://, ftp://</code>
8. Target enumeration	Map reachable internal assets	Iterate known internal hosts/ports	Intruder cluster bomb	Fuzzer	Template sweep	internal hostnames, port list
9. Impact demo	Full attacker loop	Extract creds/data, verify use	Repeater	Manual	n/a	metadata path, internal API calls

6. Evidence to collect

- Tester side: reflected internal content (basic); Collaborator/interactsh timestamp-correlated hit (blind); extracted (redacted) cloud creds + proof of live use; internal port/service response-time table; OOB side-effect proof for protocol smuggling
- Defender/forensic side: app/access logs (malicious req), egress firewall/proxy logs (outbound to metadata/internal IP), cloud audit logs (CloudTrail — credential use), DNS resolver logs (OAST domain queries), WAF logs (blocked encoded payloads), internal service logs (unexpected connection source)
- Strongest report = full chain: app log → proxy log → OOB/reflected proof → cloud audit log credential use

7. Good PoC structure

- Request/response pair showing param + trigger
- Collaborator/interactsh interaction log w/ timestamp + unique ID correlation
- Reflected internal content or metadata JSON (creds truncated/redacted)
- Evidence credential is live (1 authenticated call, if in scope)
- Internal port/service map table (open/closed/filtered signatures)
- Business impact statement tying it together

8. Attack chains

Feed INTO SSRF

- XXE → SSRF (entity fetch = SSRF delivery)
- Open redirect → SSRF (validated hop, unvalidated redirect target)
- CSRF → SSRF (trigger from victim's authenticated session)
- File upload (SVG/DOCX XXE) → SSRF

SSRF feeds INTO

- SSRF → IDOR/broken access control (internal APIs weakly authorized)
- SSRF → credential theft → cross-service/account compromise
- SSRF → RCE (gopher → Redis module write / internal Jenkins trigger)
- SSRF → internal enumeration → secondary target selection
- SSRF → session/token theft (internal admin panel reflected)
- SSRF → CORS misconfig exploitation (internal API trusts null/wildcard origin)
- SSRF → cloud infra takeover (writable IAM role, user-data secrets)

Core one-liner: SSRF breaks the assumption "only trusted internal callers reach this" — violates it simultaneously for internal APIs, internal panels, and cloud metadata.

9. False positives

- Shared HTTP client wrapper used by both tainted and system-internal calls (SAST)
- CDN/reverse-proxy health checks to internal IPs (network logs)
- Legit internal microservice-to-microservice calls (not attacker-influenced)
- DNS-based service discovery (Consul/K8s) resolving internal IPs normally
- Scanner/monitoring tool noise (Nessus/Qualys/own Nuclei runs) — exclude known scanner IPs/UAs

- Timing-only "confirmation" without OOB corroboration (jitter/GC pause mimics hang)
- Sandboxed/isolated fetch-proxy service reaching external hosts by design
- CI/CD/IaC legitimately querying 169.254.169.254 for deploy automation

10. Prerequisites

- Discoverable server-side fetch feature (no feature = no bug)
- Ability to reach the endpoint (often no special privilege needed, even unauth)
- Recon: knowledge of cloud provider / internal architecture (helps target selection, not mandatory)
- OOB listener (Collaborator/interactsh) — hard requirement for blind SSRF confirmation
- Standard tooling (Burp/ZAP/Nuclei) — efficiency, not strictly required

11. Attacker gains

- Cloud IAM credentials (metadata) → broad cloud resource access
- Internal API data, source/config files (`file://`)
- Internal network topology map (port-scan recon)
- Pivot point bypassing network segmentation entirely
- Access to internal mgmt interfaces (K8s API, internal CI/CD)
- Protocol smuggling → Redis/Memcached/SMTP interaction → potential RCE
- Regulatory exposure if PII/PCI/PHI reached
- Amplification: single SSRF → org-wide compromise via chained creds

12. CIA impact table

CIA	Value	When
Confidentiality	None	Sink exists, nothing sensitive reachable
	Low	Recon only (port/service enum)
	High	Metadata creds / internal API data / file read
Integrity	None	Read-only GET-based SSRF
	Low	Scoped write (contained blast radius)
	High	Gopher write to Redis / creds used to modify cloud resources
Availability	None	Purely informational fetch
	Low	Minor resource exhaustion, self-recovering
	High	Internal service crash/DoS or cloud resource deletion via stolen creds

Scope: almost always **Changed (S:C)** — impacts resources outside vulnerable component's own authority (common interview trap if left Unchanged).

13. CVSS vectors by scenario

Scenario	Vector	Score	Sev
Blind SSRF, recon only	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	5.3	Medium
Basic SSRF, internal data exposed	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N	9.3	Critical
Metadata credential theft	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:L/A:N	9.6	Critical
SSRF → RCE (gopher)	AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H	9.4	Critical
SSRF via CSRF delivery	AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N	8.7	High

14. Business impact (financial app context)

- Cloud account compromise → access to payment/ledger data stores beyond app scope
- PCI-DSS: CDE compromise → loss of processing privileges, fines
- GDPR/HIPAA-adjacent: customer PII/financial record exposure → breach notification obligations
- Bypasses network segmentation investment → invalidates other controls' assumptions org-wide
- Blast radius harder to bound than typical app-layer bug → expensive IR (credential rotation, audit log tracing)
- Reputational: "cloud metadata SSRF" pattern is a well-known headline-class disclosure

15. IOCs / signals

- Outbound conns to RFC1918/loopback from app tier
- Requests to 169.254.169.254 (near-100% signal)
- DNS queries to OAST domains (*.burpcollaborator.net, *.oastify.com, *.interact.sh)
- Unusual outbound ports (22, 3306, 6379, 9200, 2379) from web frontend
- Encoded IPs in WAF/proxy logs (decimal/octal/hex)
- Rapid varying host/port in single param from one source IP (scan signature)
- Non-HTTP schemes submitted to fetch endpoints
- Timing/error anomalies correlated to param values

16. Root cause / vulnerable code

- Untrusted input → HTTP client sink, no destination validation
- OR validation checks wrong thing (string hostname, not resolved IP) / checked too early (pre-redirect, pre-second-DNS-resolution)

```
# vulnerable - no check
def fetchUserContent(url):
    return httpClient.get(url)

# vulnerable - blocklist bypassed by encoding/rebinding
def fetchUserContent(url):
    if "localhost" in url or "127.0.0.1" in url: reject()
    return httpClient.get(url) # 0x7f000001, [::1], decimal IP bypass

# vulnerable - DNS rebinding gap
def fetchUserContent(url):
    ip = resolveDns(parseHost(url))
    if not isAllowed(ip): reject()
    return httpClient.get(url) # resolves DNS again internally -> different IP
```

17. Remediation / secure logic

- Allowlist hosts/domains (closed-world), not blocklist
- Restrict scheme to http/https explicitly
- Validate resolved IP (not hostname string) — block private/loopback/link-local ranges
- Pin resolved IP through to actual connection (no second DNS lookup) — closes rebinding gap
- Disable redirect following by default, or re-validate on each hop

```
ALLOWED_HOSTS = {"api.trusted-partner.com"}

def safe_fetch(url):
    p = urlparse(url)
    if p.scheme not in ("http", "https"): reject()
    if p.hostname not in ALLOWED_HOSTS: reject()
    ip = ipaddress.ip_address(socket.gethostbyname(p.hostname))
    if ip.is_private or ip.is_loopback or ip.is_link_local: reject()
    return requests.get(url, allow_redirects=False, timeout=3)
```

- Java: `Redirect.NEVER`, `InetAddress.isSiteLocalAddress/isLoopbackAddress`
- Node: `maxRedirects:0`, `dns.lookup()` + private-range check before fetch
- C#: `AllowAutoRedirect=false`, `Dns.GetHostAddresses` + range check

18. Compensating controls

- Network egress deny from app tier to internal mgmt/admin ranges
- IMDSv2 enforcement (token-based metadata access) / disable metadata svc where unneeded
- Dedicated network-isolated fetch-proxy microservice for all user-URL fetches
- Network-layer DNS pinning
- WAF signatures for encoded-IP/gopher/file patterns (weak, defense-in-depth only)
- Response sanitization — strip credential-like tokens before rendering fetched content
- Least-privilege IAM role scoping (limits Confidentiality→Integrity/Availability escalation)

- Shared safe-fetch wrapper library + SAST taint rule enforcement

19. Repeated occurrence → posture signal

- Indicates missing shared/vetted HTTP-fetch abstraction — devs reaching for raw client libs per-feature
- No upstream design checklist item for "does this feature make server-side requests from user input"
- SAST not covering HTTP client sinks / sanitizer not enforced org-wide
- Same systemic framing as recurring IDOR — control failure, not isolated dev mistake; relevant for exec/regulatory reporting

20. Upstream improvements

- Shared `safeFetch()` / `SafeUrlFetcher` wrapper as the only sanctioned HTTP-egress path
- Semgrep/CodeQL taint rule: source (request params/body/headers) → sink (HTTP client libs) → sanitizer (wrapper) — staged WARNING→BLOCKING, diff-aware
- Framework-level protections: disable auto-redirect by default in HTTP client config templates
- Design checklist item: "does this feature fetch a URL server-side? If yes → must use `safeFetch()`"
- IMDSv2 enforced by default in cloud infra baseline
- Network segmentation as default (deny-by-default egress) not opt-in

21. Sample triage response

Confirmed SSRF on `POST /api/import?url=`. Verified via Burp Collaborator OOB DNS/HTTP interaction, timestamp-correlated to test request [ref: request #X]. Escalated to internal target — retrieved AWS instance metadata (IAM role name confirmed, full credential redacted per scope). Scope=Changed, Confidentiality=High. Severity: Critical (CVSS 9.6). Recommend immediate: (1) rotate exposed instance role credentials, (2) confirm IMDSv2 enforcement, (3) patch per remediation note. Full attacker-loop evidence attached (request/response, Collaborator log, redacted credential proof).

22. Sample developer remediation note

`import_url` param (`api/import.py:42`) passed directly to `requests.get()` with no destination validation — CWE-918. Replace direct call with `safe_fetch()` wrapper (allowlist + resolved-IP validation + `allow_redirects=False`, see `utils/http_safe.py`). Do not blocklist strings — use allowlist. Add unit test asserting request to `169.254.169.254` and `127.0.0.1` raises `ValueError`. SAST rule `ssrf-tainted-url-to-http-client` will flag any bypass of the wrapper going forward.

23. Real-world reference

- **Capital One breach (2019)**: SSRF against a misconfigured WAF/EC2 instance → retrieved instance metadata IAM credentials → used to access and exfiltrate ~100M+ customer records from S3. Textbook metadata-SSRF-to-mass-data-breach chain; frequently cited in interviews for business-impact framing.
- General pattern: numerous HackerOne/Bugcrowd reports on webhook/PDF-export/link-preview features hitting `169.254.169.254` — recurring root cause across bug bounty programs.